Vidyavardhini's College of Engineering & Technology

Department of Computer Science Engineering (Data Science)

Academic Year: 2026-27

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2026-27

Course: Wireless and Mobile Communication
Course code: **Year: TE**
SEM: V

Experiment No. 3
Aim: To demonstrate the concept of frequency reuse in cellular systems using Python/MATLAB-based simulation and visualization.
Name: Meet Mehta
Roll Number: 30
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Mrs. Sonali Sable

Signature : **Date** :

Aim: To demonstrate the concept of frequency reuse in cellular systems using Python/MATLAB-based simulation and visualization.

Objectives

1. **To understand the concept of cellular frequency reuse.**
2. **To understand the formation of a cellular cluster.**
3. **To assign frequency groups A–G to a 7-cell cluster.**
4. **To distinguish between correct frequency reuse and incorrect frequency assignment.**
5. **To observe the effect of frequency reuse distance on co-channel interference.**
6. **To visualize co-channel interference when reused frequency groups are placed too close to each other.**

Theory:

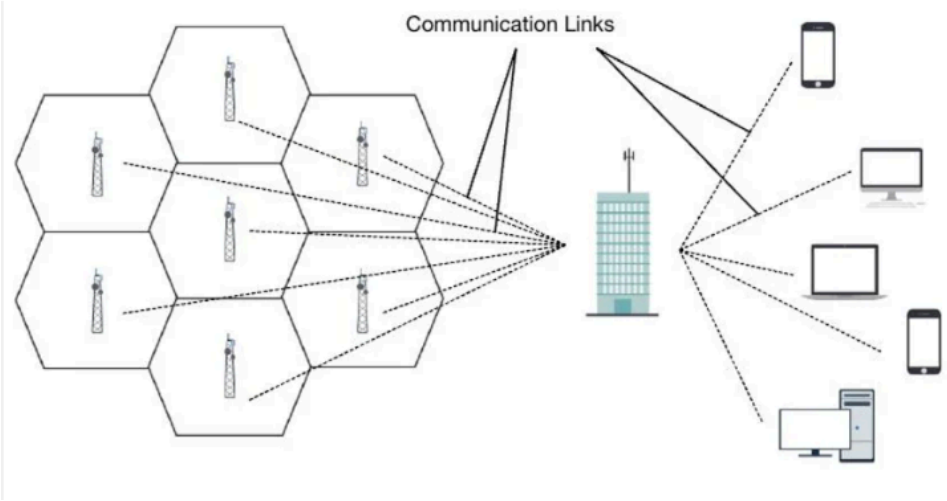
1. Cellular Communication System

A cellular communication system divides a large geographical area into smaller regions called cells.

Each cell is served by a base station, which provides wireless communication to users within that cell.

The base stations establish communication links with mobile users and connect them to the communication network.

The same geographical area can therefore be covered by multiple interconnected cells



The diagram illustrates how multiple cells, their base stations, and communication links work together to provide wireless connectivity to users over a large geographical area.

Frequency Reuse:

Frequency reuse is the technique of using the same frequency groups in different cells that are sufficiently separated from each other.

For example, suppose the available frequency groups are:

A, B, C, D, E, F, G

A group of cells in which each frequency group A–G is used only once forms a cluster.

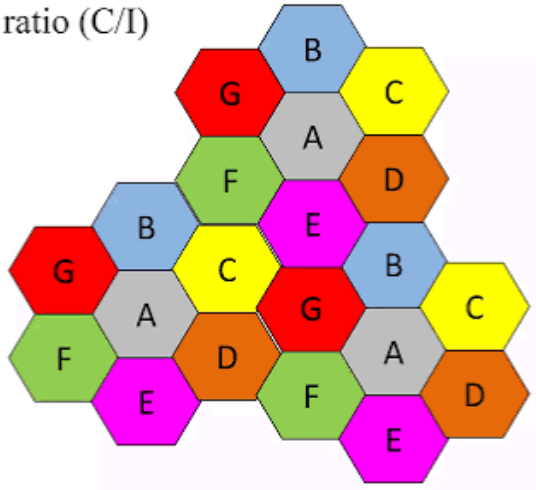
The same frequency groups are then reused in other cells or clusters at an appropriate reuse distance, as illustrated in the diagram.

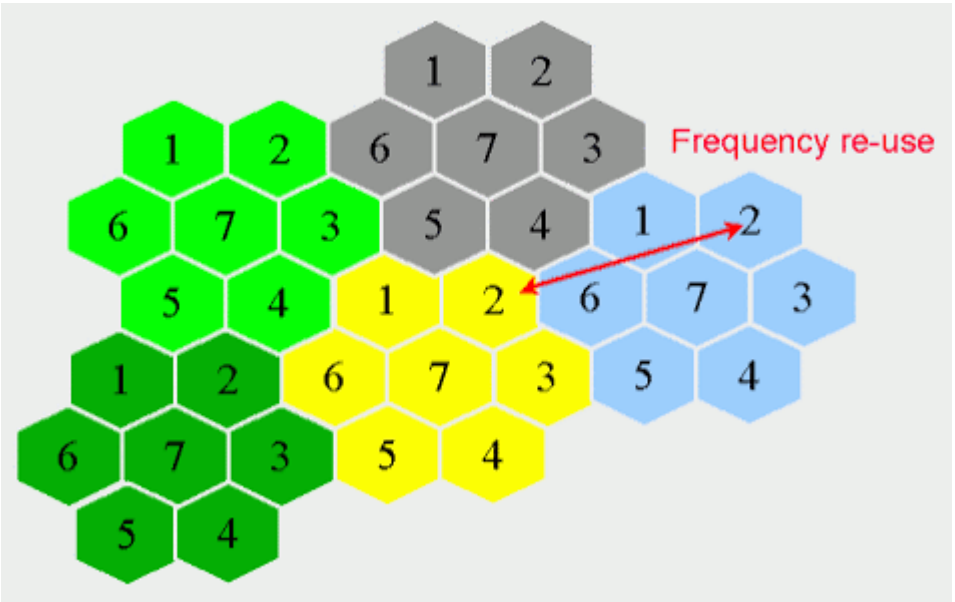
In the diagram, cells marked with the same letter use the same frequency group. For example, cells marked A use the same frequency group, cells marked B use the same frequency group, and so on.

The repeated frequency groups are separated geographically to control co-channel interference.

Frequency Reuse...

- Cells with same letter use the same set of frequency channels
- Using hexagonal cells, BS located at center of cell
- MS at edge of cell receives weak signal from BS, i.e., low Carrier to Interference ratio (C/I)





Cluster Size

A cluster is a group of cells in which each available frequency group is used only once.

The number of cells in a cluster is called the cluster size, represented by N.

The commonly used cluster sizes are:

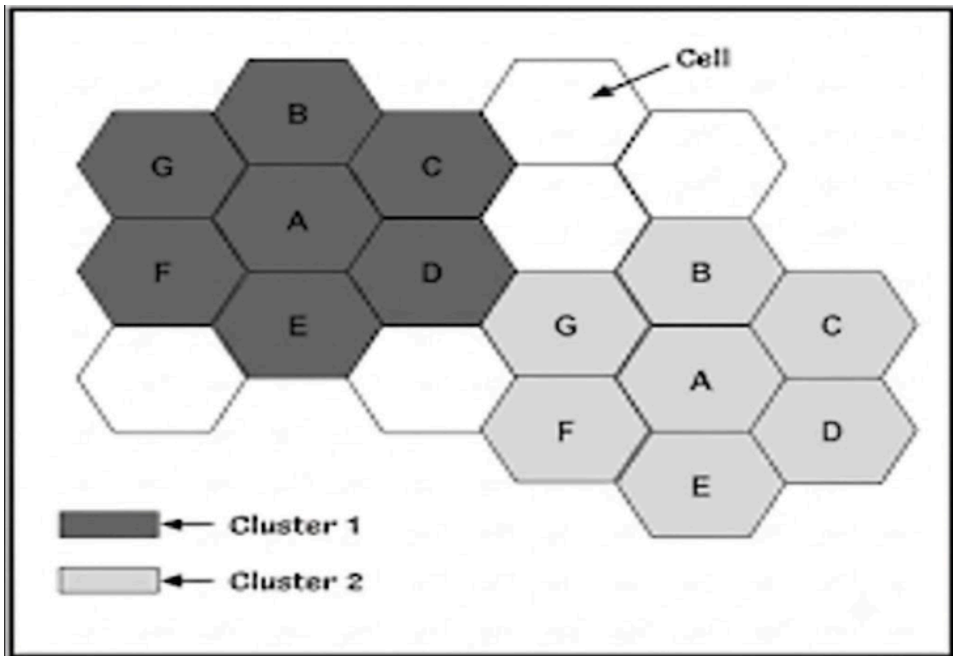
N = 1, 3, 4, 7, 9, 12, 13, …

For this experiment, a 7-cell cluster is used because it provides a simple and clear example of frequency assignment and frequency reuse.

In a 7-cell cluster, seven different frequency groups are assigned to the seven cells, with each frequency group used only once.

For example:

A, B, C, D, E, F, G



The diagram shows two clusters, each containing seven cells. The seven frequency groups are assigned once within each cluster, and the same frequency pattern can be reused in another cluster.

Correct and Incorrect Frequency Assignment

1. **Correct Frequency Assignment**

Within a 7-cell cluster, each frequency group is assigned only once.

For example:

A – B – C – D – E – F – G

Here, no frequency group is repeated within the cluster.

2. **Incorrect Frequency Assignment**

If the same frequency group is assigned to two cells within the same cluster, it is an incorrect frequency assignment.

For example:

A – A – B – C – D – E – F

Here, A is repeated within the same cluster.

This is not frequency reuse because frequency reuse requires the same frequency group to be used in a different, geographically separated cell or cluster.

Important Rule

Same cluster + same frequency group repeated = Incorrect frequency assignment

Different sufficiently separated clusters + same frequency group reused = Frequency reuse

This distinction should also be demonstrated separately in the Python/MATLAB simulation.

Frequency Reuse Distance

The distance between the centers of two nearest cells using the same frequency is called the frequency reuse distance, represented by D.

The relationship is:

RD=3N

where:

- **D = frequency reuse distance**
- **R = radius of a cell**
- **N = cluster size**

Therefore,

It should be:

$$D = R\sqrt{3N}$$

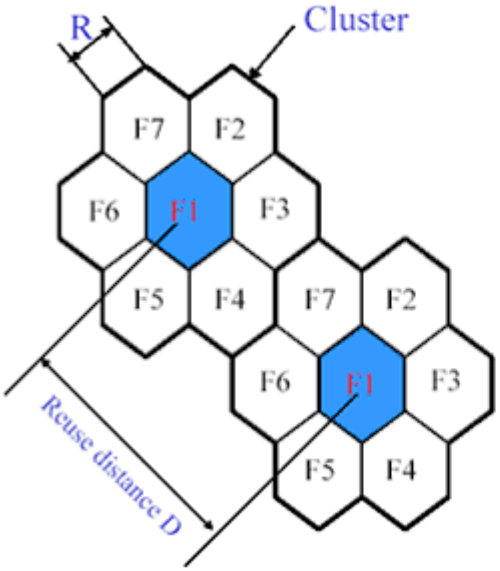
where:

- **D = frequency reuse distance**
- **R = cell radius**
- **N = cluster size**

For $N = 7$:

$$D = R\sqrt{3(7)}D = R\sqrt{21}D \approx 4.58R$$

Thus, two cells using the same frequency should be sufficiently separated to reduce interference.



For hexagonal cells, the reuse distance is given by

$$D = \sqrt{3N} \times R$$

where R is cell radius and N is the reuse pattern (the cluster size or the number of cells per cluster)

Reuse factor is

$$\frac{D}{R} = \sqrt{3 \times N}$$

Co-Channel Interference

When two cells use the same frequency group, they are called co-channel cells.

For example:

Cell 1 → A

Cell 8 → A

These two cells are co-channel cells because both use frequency group A.

If the co-channel cells are sufficiently far apart, the interference can be controlled.

However, if the same frequency group is reused in cells that are too close to each other, their signals can interfere with each other. This is known as co-channel interference.

Important Concept

Correct frequency reuse does not itself mean that interference will necessarily be high.

The problem occurs when the reuse distance is insufficient.

Execution Steps

Step 1: Run the Python/MATLAB Frequency Reuse Simulation

Run the Python or MATLAB program developed for the frequency reuse experiment.

The simulation should generate or display a cellular network consisting of hexagonal cells.

The simulation may allow you to:

- **Create multiple hexagonal cells.**
- **Assign frequency groups to cells.**
- **Display the frequency assignment pattern.**
- **Create multiple clusters.**
- **Reuse frequency groups in another cluster.**
- **Change the distance between clusters.**
- **Observe the effect of co-channel interference.**

What to Observe

Initially identify:

- **Number of cells**
- **Cell boundaries**
- **Base station locations, if represented**
- **Frequency groups assigned to the cells**
- **Distance between cells/clusters**

Note: The exact appearance and available features may vary depending on whether Python or MATLAB is used.

Step 2: Observe the Cellular Network and Form a 7-Cell Cluster

Observe the hexagonal cellular network generated by the simulation.

Each hexagonal cell represents a geographical coverage area served by a base station.

For this experiment, identify a group of seven cells and consider it as the first cluster.

The seven cells will be assigned the frequency groups:

A, B, C, D, E, F, G

The hexagonal arrangement may vary depending on the Python/MATLAB implementation.

Identify in the simulation:

1. **Cell boundaries**
2. **Base station locations, if shown**
3. **Frequency group assigned to each cell**
4. **Neighboring cells**
5. **Cells belonging to the same cluster**

Step 3: Assign Frequency Groups A–G and Demonstrate Correct/Incorrect Assignment

Assign seven different frequency groups to the seven cells of the first cluster.

Cell	Frequency Group
Cell 1	A
Cell 2	B
Cell 3	C
Cell 4	D
Cell 5	E
Cell 6	F
Cell 7	G

Therefore, the first cluster contains:

A, B, C, D, E, F, G

Important Rule

Each frequency group must be assigned only once within a cluster.

Now perform an incorrect assignment test by assigning the same frequency group to two cells within the same cluster.

For example:

A, A, B, C, D, E, F

Here, frequency group A is repeated within the same cluster.

This represents an incorrect frequency assignment.

Repeating the same frequency group within the same cluster is not frequency reuse.

Restore the correct assignment A–G before proceeding to the next step.

Step 4: Create a Second Cluster and Reuse Frequency Groups

After correctly assigning A–G to the first cluster, create a second 7-cell cluster at a suitable distance from the first cluster.

Assign the same frequency groups to the second cluster.

First Cluster	Second Cluster
A	A
B	B
C	C
D	D
E	E
F	F
G	G

For example:

- **Cell 1 → A**
- **Cell 8 → A**

- **Cell 2 → B**
- **Cell 9 → B**
- **Cell 3 → C**
- **Cell 10 → C**

and so on.

Observe that frequency group A is used once in the first cluster and reused in the second cluster.

The same applies to B, C, D, E, F, and G.

This is the correct frequency reuse condition: the same frequency groups are reused in a different cluster with sufficient separation.

Step 5: Identify the Reuse Pattern and Calculate Reuse Distance

Observe the complete network and identify cells using the same frequency group.

For example:

Frequency Group	Cells Using It
A	Cell 1, Cell 8
B	Cell 2, Cell 9
C	Cell 3, Cell 10
D	Cell 4, Cell 11
E	Cell 5, Cell 12
F	Cell 6, Cell 13
G	Cell 7, Cell 14

You should be able to identify the repeated frequency groups:

A → A

B → B

C → C

etc.

Now measure or obtain the distance between the centers of corresponding co-channel cells, depending on the features of the Python/MATLAB simulation.

The theoretical frequency reuse distance is:

$$D = R\sqrt{3N}$$

where:

- **D = Frequency reuse distance**
- **R = Cell radius**
- **N = Cluster size**

For N = 7:

$$D = R\sqrt{21} D \approx 4.58R$$

Compare the theoretical reuse distance with the distance represented in the simulation, if the simulation provides the required measurements.

Step 6: Demonstrate Co-Channel Interference by Reducing Reuse Distance

Now use the correctly assigned two clusters from Step 4.

Gradually move the second cluster closer to the first cluster in the Python/MATLAB simulation.

Observe the corresponding co-channel cells, such as:

A in Cluster 1 ↔ A in Cluster 2

Initially, keep the clusters sufficiently separated.

Then reduce the separation between the clusters so that the corresponding co-channel cells become too close to each other.

Observe the effect

When the reuse distance becomes insufficient, co-channel interference may increase.

Depending on the Python/MATLAB simulation, you may observe:

- **Increased interference**
- **Reduced signal quality**
- **Lower SINR**

- **Signal distortion**
- **Increased error rate**
- **Reduced communication quality**

The important point is that the frequency assignment remains correct, but the co-channel cells are now too close because the reuse distance has been reduced.

Thus:

Correct frequency reuse + sufficient distance → controlled interference

Correct frequency reuse + insufficient distance → increased co-channel interference

This is a much better demonstration than changing two neighboring cells in the same cluster to A.

Step 7: Record the Observations

Record the results obtained from the Python/MATLAB simulation in the following observation table.

Python/ MATLAB code:

```
import numpy as np

import matplotlib.pyplot as plt

from matplotlib.patches import RegularPolygon


def get_hex_center(q, r, radius=1.0):

    x = radius * np.sqrt(3) * (q + r / 2.0)

    y = radius * 1.5 * r

    return x, y


def create_three_clusters(radius=1.0):

    cluster_offsets = [

        (0, 0),      # Cluster 1 (Bottom Left)

        (3, -1),     # Cluster 2 (Bottom Right)

        (1, 3)       # Cluster 3 (Top Middle)
```

```
    ]
    local_cells = [
        (0, 0, 'A'),
        (1, 0, 'B'),
        (0, 1, 'C'),
        (1, 1, 'D')
    ]
    all_cells = []
    cell_id = 1
    for cluster_idx, (cq, cr) in enumerate(cluster_offsets):
        for dq, dr, group in local_cells:
            q = cq + dq
            r = cr + dr
            x, y = get_hex_center(q, r, radius)
            all_cells.append({
                'id': cell_id,
                'cluster': cluster_idx + 1,
                'group': group,
                'x': x,
                'y': y
            })
            cell_id += 1
    return all_cells


def plot_tight_clusters(cells, title):
    fig, ax = plt.subplots(figsize=(10, 8))
    ax.set_aspect('equal')
```

```
group_colors = {
    'A': '#FF9999',
    'B': '#66B2FF',
    'C': '#99FF99',
    'D': '#FFCC99'
}
for cell in cells:
    facecolor = group_colors[cell['group']]
    hex_cell = RegularPolygon(
        (cell['x'], cell['y']),
        numVertices=6,
        radius=1.0,
        orientation=0,
        facecolor=facecolor,
        edgecolor='black',
        linewidth=1.5,
        alpha=0.85
    )
    ax.add_patch(hex_cell)
    ax.text(cell['x'], cell['y'] + 0.15, f"Cell {cell['id']}",
            ha='center', va='center', weight='bold', fontsize=8)
    ax.text(cell['x'], cell['y'] - 0.05, f"Cluster {cell['cluster']}",
            ha='center', va='center', fontsize=7)
    ax.text(cell['x'], cell['y'] - 0.25, f"Group: {cell['group']}",
            ha='center', va='center', weight='bold', fontsize=8)
ax.autoscale_view()
plt.title(title, fontsize=14, weight='bold')
```

```
    plt.xlabel("X Distance")
    plt.ylabel("Y Distance")
    plt.grid(False)
    plt.show()


cells_data = create_three_clusters(radius=1.0)
plot_tight_clusters(
    cells_data,
    title="3 Contiguous Clusters (N=4, Groups A-D) with Reuse Distance"
)


incorrect_cells_data = create_three_clusters(radius=1.0)
incorrect_cells_data[6]['group'] = 'D'
incorrect_cells_data[7]['group'] = 'C'


def plot_tight_clusters_cci(cells, title, cci_cell_ids=[]):
    fig, ax = plt.subplots(figsize=(10, 8))
    ax.set_aspect('equal')
    group_colors = {
        'A': '#FF9999',
        'B': '#66B2FF',
        'C': '#99FF99',
        'D': '#FFCC99'
    }
    for cell in cells:
        facecolor = group_colors[cell['group']]
        edgecolor = 'black'
```

```
    linewidth = 1.5
    if cell['id'] in cci_cell_ids:
        edgecolor = 'red'
        linewidth = 3.5
    hex_cell = RegularPolygon(
        (cell['x'], cell['y']),
        numVertices=6,
        radius=1.0,
        orientation=0,
        facecolor=facecolor,
        edgecolor=edgecolor,
        linewidth=linewidth,
        alpha=0.85
    )
    ax.add_patch(hex_cell)
    ax.text(cell['x'], cell['y'] + 0.15, f"Cell {cell['id']}",
            ha='center', va='center', weight='bold', fontsize=8)
    ax.text(cell['x'], cell['y'] - 0.05, f"Cluster {cell['cluster']}",
            ha='center', va='center', fontsize=7)
    ax.text(cell['x'], cell['y'] - 0.25, f"Group: {cell['group']}",
            ha='center', va='center', weight='bold', fontsize=8)
ax.autoscale_view()
plt.title(title, fontsize=14, weight='bold')
plt.xlabel("X Distance")
plt.ylabel("Y Distance")
plt.grid(False)
plt.show()
```

```
plot_tight_clusters_cci(
    incorrect_cells_data,
    title="Incorrect Allocation: Interchanged Group C & D in Cluster 2 Causing CCI",
    cci_cell_ids=[4, 7]
)
```

OUTPUT :

Figure 1: Correct frequency reuse using three N=4 clusters.

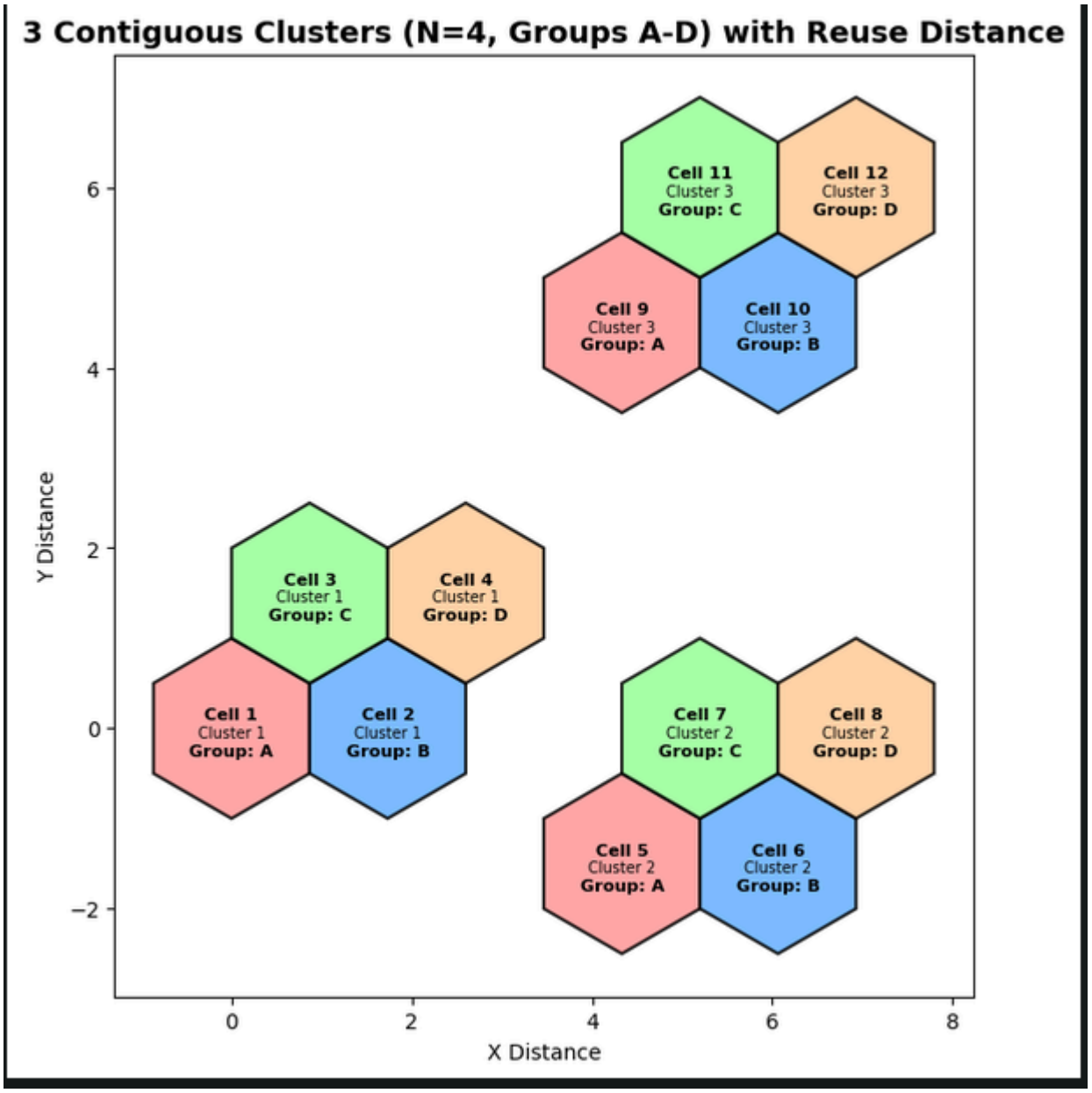
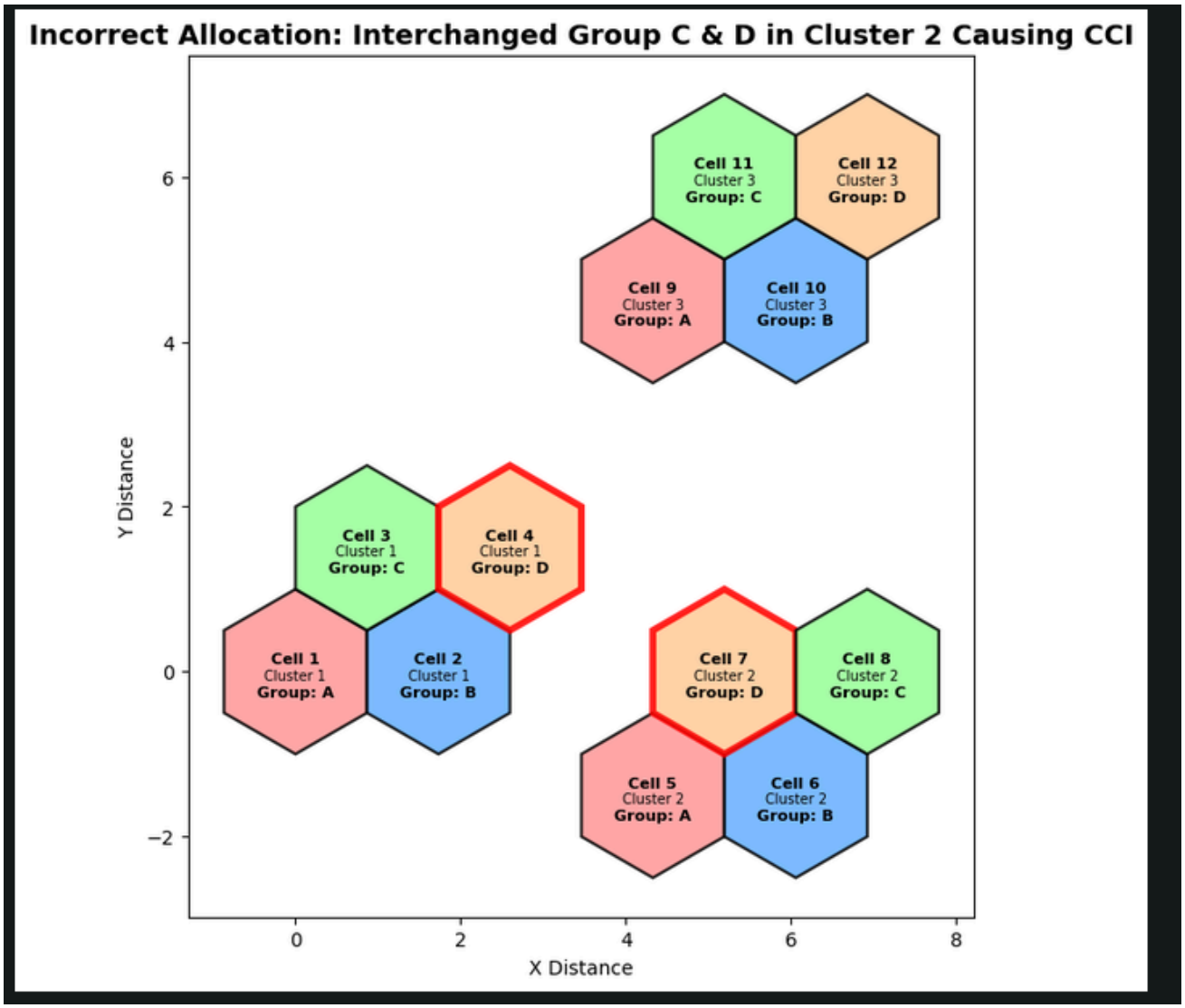


Figure 2: Incorrect allocation showing CCI between cells 4 and 7.



Observation Table:

Cell Number	Frequency Group	Adjacent Cells Using Same Frequency (Yes/No)	Frequency Reused (Yes/No)	Observation
1	A	No	Yes	Correct Reuse
2	B	No	Yes	Correct Reuse
3	C	No	Yes	Correct Reuse
4	D	No	Yes	Correct Reuse
5	A	No	Yes	Reused Group
6	B	No	Yes	Reused Group
7	C	No	Yes	Reused Group
8	D	No	Yes	Reused Group
9	A	No	Yes	Reused Group
10	B	No	Yes	Reused Group

11	C	No	Yes	Reused Group
12	D	No	Yes	Reused Group

Note: Record the observations for each cell individually in the observation table, ensuring that all entries match the output generated by the Python/MATLAB simulation.

Conclusion: Thus, the concept of frequency reuse in cellular communication was demonstrated using Python visualization. Multiple hexagonal clusters were created and frequency groups were reused between clusters. The simulation also demonstrated that incorrect frequency allocation or insufficient reuse distance can result in co-channel interference. Hence, proper frequency assignment and sufficient reuse distance are necessary to control interference and maintain communication quality

Viva Questions

What is frequency reuse?
Frequency reuse is the technique of using the same frequency group in different cells that are sufficiently separated.

What is a cluster?
A cluster is a group of cells in which each available frequency group is used only once.

What is the difference between frequency reuse and incorrect frequency assignment?
Frequency reuse uses the same frequency in different sufficiently separated clusters. Incorrect assignment repeats the same frequency within the same cluster.

What is frequency reuse distance?
It is the distance between the centers of two nearest cells using the same frequency.

What is co-channel interference?
It is interference between cells using the same frequency when they are too close.

Why is frequency reuse necessary?
It allows limited frequency resources to be reused across different geographical areas.

What happens if the reuse distance is too small?
Co-channel interference increases, which can reduce signal quality and communication performance.